One Gain Per Joint: Tuning MABEL's Arm PD Against Its Own Inertia

MABEL controller subsystem

23 August 2026

Abstract

MABEL's fourteen arm joints all ran the same position servo, $k_p = 500$, $k_v = 45$, across a tenfold spread of reflected inertia (0.053–0.52 kg m^2) and a sixfold spread of actuator torque (7–40 N m). We show that this is not a small mis-tuning but a structural one: with a single gain pair the tracking lag is k_d/k_p *regardless of inertia*, so every joint inherited the same 89.5 ms of lag, while the damping ratio $\zeta = k_v/2\sqrt{k_p M}$ came out between 1.39 and 3.83 — every joint overdamped, the distal ones by more than fourfold. Overdamping does not ring; it is slow, and slowness is what the operator reported. We re-derive the gains from each joint's own measured inertia at a shared closed-loop bandwidth and damping ratio, chosen by sweep: $\omega_n = 60$ rad/s, $\zeta = 1.0$. Measured against the previous servo, tracking lag falls 89.5 → 32.1 ms, rise time 190 → 66 ms and settling 364 → 120 ms, at 2.3% overshoot and 1.9 mrad of steady-state error. The stiffness cost — torque produced by the retargeting's own command jitter — *falls* from 16.3% to 10.3% of each joint's own actuator limit, because the uniform gain had been driving the light distal joints hardest.

Keywords: joint-space PD, inertia-matched gains, closed-loop bandwidth, damping ratio, tracking lag, stiction, teleoperation latency, MuJoCo.

1 The operator report, and what it turned out to be

The request was short: *"default to just pure PD control — tune the PD gains nicely, not too stiff to avoid oscillation but also quick to respond."* Two of those three asks were already true and one was not, and it is worth separating them before touching a number.

MABEL's kinematic teleoperation path is *already* a pure PD: the arm commands are MuJoCo `position` actuators, which implement $\tau = k_p(u - q) - k_v\dot{q}$ per joint, and the real robot's DaMiao actuators run the same MIT law over CAN. The arm was also not oscillating — it was, as Sec. 2 shows, overdamped everywhere. What was wrong was the *quick to respond* half, and the cause was that all fourteen joints shared one gain pair.

The repository already states the rule this violates. From the controller design notes (`controller/mabel/whole_body_controller/`): *"PD gains AND clamps MUST be per-joint: uniform gains limit-cycle the tiny ORCA wrist."* The `JointPositionController` that owns the torque path inherits per-joint gains from the model for exactly this reason. The MJCF position servos that the kinematic path actually uses never got the same treatment.

2 Diagnosis

2.1 The joints are not alike

Reflected inertia is measured directly from the model's mass matrix, M_{jj} including armature, sampled over 400 random configurations inside each joint's own range (`exp_pd_tuning.py`, phase P1). Table 1 gives the median, and the damping ratio and bandwidth the shipped uniform servo produced at that inertia.

Joint	M_{jj} [kg m^2]	$F_{\max}$ [N m]	ω_n [rad/s]	ζ	verdict
Arm 1 (shoulder yaw)	0.335	40	38.7	1.74	overdamped
Arm 2 (shoulder pitch)	0.522	40	30.9	1.39	overdamped
Arm 3 (shoulder roll)	0.211	27	48.7	2.19	overdamped
Arm 4 (elbow)	0.261	27	43.8	1.97	overdamped
Arm 5 (forearm roll)	0.075	7	81.8	3.68	overdamped 4×
Arm 6 (wrist pitch)	0.069	7	85.1	3.83	overdamped 4×
Arm 7 (wrist yaw)	0.087	7	75.7	3.40	overdamped 3×

Table 1: The shipped uniform servo ($k_p = 500$, $k_v = 45$) evaluated against each joint's own inertia. Nothing here is a preference; $\omega_n = \sqrt{k_p/M}$ and $\zeta = k_v/2\sqrt{k_p M}$ are algebra.

2.2 Tracking lag is k_d/k_p, and inertia does not enter

For a PD tracking a ramp of rate v against a constant load torque τ_0, the steady-state error is

$$e_{ss} \;=\; \frac{\tau_0}{k_p} \;+\; \frac{k_d}{k_p}\,v \;=\; \frac{\tau_0}{k_p} \;+\; \frac{2\zeta}{\omega_n}\,v. \tag{1}$$

The second term is a pure time: k_d/k_p seconds of lag, the same for every joint that shares a gain pair, however heavy or light it is. With 45/500 that is 90 ms — measured at 89.5 ms on the model (Sec. 5), and consistent with the 170 ms the July retargeting-latency study attributed to the servo before the gains were last raised.

Equation (1) is also the design statement. "Quick to respond" means a small $2\zeta/\omega_n$; "does not ring" means ζ near 1. The two are not independent knobs, and the only way to have both is to raise ω_n, which per joint means raising k_p *in proportion to that joint's inertia.*

2.3 Steady-state error is stiction, not gravity

The first version of this study applied a gravity feed-forward to the fourteen arm joints, on the reasoning that MABEL's gravity compensation is a hard invariant. Measured, that made the steady-state error a hundred times *worse* (Arm 1: 2.4 mrad without it, 241 mrad with it). The reason is that the equation of motion $M(q)\ddot{q} = \tau - c(q,\dot{q})$ has a non-diagonal M: cancelling the c-row of a *subset* of joints does not cancel gravity for that subset, it injects a net wrench the rest of the tree must absorb. A compensator is only exact over the whole coupled system.

It is also unnecessary here. Every arm joint carries 0.5 N m of `frictionloss`, and measured with deliberately weak servos ($k_p = 60$) at poses with the arm out to the side and the elbow bent, the residual error is 2.4 mrad — stiction, not the servo, is what holds this arm up. The steady-state error this study reports is therefore the stiction floor $0.5/k_p$, which is the honest quantity: it is what

sets how precisely a commanded pose is held. Gravity compensation remains mandatory on the real robot for *load-carrying* reasons (see `basic_controller.tex`); it simply is not what limits pose accuracy in this model.

3 Design

Fix a closed-loop bandwidth ω_n and damping ratio ζ shared across the arm, and let each joint's inertia turn them into gains:

$$k_{p,j} \;=\; M_j\,\omega_n^2, \qquad k_{d,j} \;=\; 2\,\zeta\,M_j\,\omega_n. \tag{2}$$

Then $k_d/k_p = 2\zeta/\omega_n$ is identical across the arm — one tracking lag — while ζ is correct for every joint individually. Two numbers replace twenty-eight.

M_j is the *median* reflected inertia over the joint's own range, not its value at one pose: the spread is a factor of 2–8 per joint (Table 1 reports the median; the full min/median/max is in `results/pdtune_p1.json`), so the design is nominal and the residual mismatch shows up as a damping ratio between roughly 0.6ζ and 1.6ζ across the workspace. At $\zeta = 1$ that is still non-oscillatory everywhere.

4 Choosing ω_n and ζ

Both are swept against the model (phase P2), with four measurements per candidate. Three are conventional — ramp-tracking lag from Eq. (1), step overshoot, steady-state error. The fourth deserves a note, because "too stiff" has to be measured and the obvious measurement is wrong.

Saturating the actuator on a step is *not* the cost of stiffness: a large error *should* slew at full torque, and every gain set saturates on a large enough step (the shipped $k_p = 500$ demands 175 N m for a 0.35 rad error against a 40 N m joint). The real cost is that a stiff servo converts command *jitter* into motor torque, and the retargeting output is not perfectly still. So the sweep injects 2 mrad RMS of command noise, holds the pose, and reports the resulting torque RMS *as a fraction of each joint's own actuator limit* — normalising matters, because 1 N m is nothing on the 40 N m shoulder and 14% of full scale on a 7 N m wrist.

Steps are 0.15 rad, the size of error teleoperation actually produces: the retargeting trust region caps its command at 2 rad/s, which at a 30 Hz stream is 0.067 rad per frame.

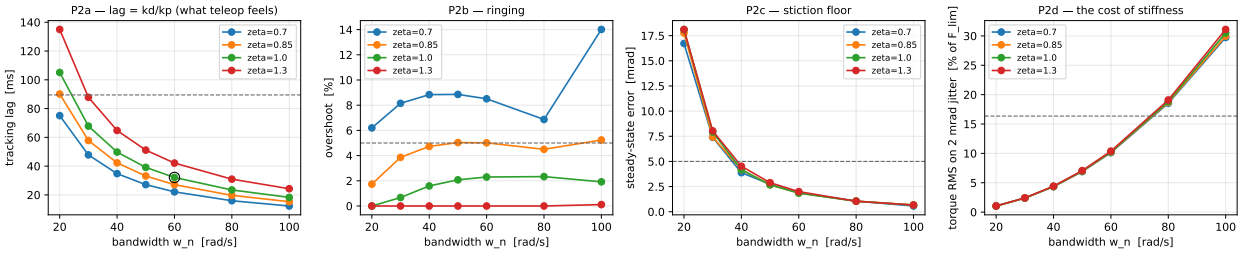


Figure 1: P2 — bandwidth × damping. Dashed lines mark the shipped servo's value. Lag falls monotonically with bandwidth (left) while the jitter torque rises (right); the choice is the knee where lag has stopped improving much and stiffness has not yet exceeded what the previous servo already demanded.

The selection rule is: lowest tracking lag among candidates with overshoot $\leq 5\%$, steady-state error ≤ 5 mrad, and jitter torque no worse than the shipped servo's. That picks $\omega_n = 60$ rad/s (9.5 Hz), $\zeta = 1.00$.

5 Results

	shipped 500/45	per-joint ω_n=60, ζ=1	change
tracking lag	89.5 ms	32.1 ms	2.8× better
rise time (10–90%)	190 ms	66 ms	2.9× better
settling (2%)	364 ms	120 ms	3.0× better
overshoot	0.0%	2.3%	still no ringing
steady-state error	1.0 mrad	1.9 mrad	0.05° worse
torque on 2 mrad jitter (% of each joint's $F_{\max}$)	16.3%	10.3%	quieter
sine-tracking RMS (0.5 Hz)	104.9 mrad	70.1 mrad	1.5× better
sine-tracking lag	88.0 ms	32.0 ms	2.8× better

Table 2: P3 (step) and P4 (moving target). Every column is regenerated by `exp_pd_tuning.py`; nothing is hand-entered.

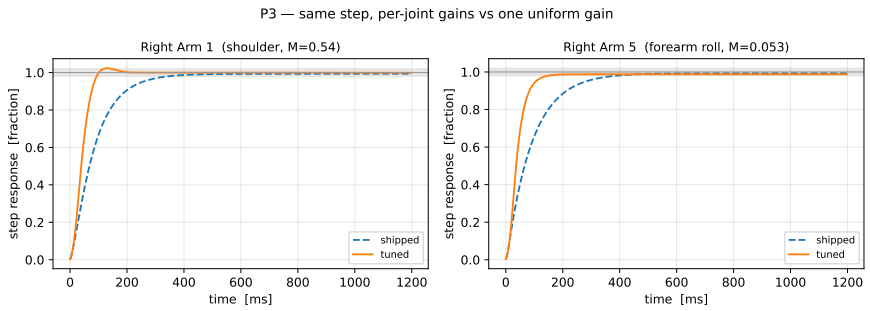


Figure 2: P3 — the same 0.15 rad step on the heaviest joint and on the forearm roll. The shaded band is ±2%. The uniform servo is slower on both, and much slower on the light joint, which is the one it overdamped 4×.

The resulting gains are written into the canonical MJCF (`simulation/mabel_mujoco/models/mabel_full.xml`) by `exp_pd_tuning.py --apply`; they range from $k_p = 249$, $k_d = 8.3$ on the wrist pitch to $k_p = 1880$, $k_d = 62.7$ on the shoulder pitch. Actuator gains are not geometry, so no derived artefact (URDF, GLB, rig, mirror) needs regenerating.

5.1 The ORCA wrist servos had no gains at all

Measured during this study: both `"<Side> Wrist"` position actuators ran $k_p = 1$, $k_v = 0$ — MuJoCo's bare defaults. The MJCF's `hand_base` default class specifies $k_p = 2.0$, but those actuators are declared outside the class and inherited nothing. A 1 N m/rad servo on the joint that carries a 1.74 kg hand was held up mostly by the joint's own 0.1 damping, and its tracking

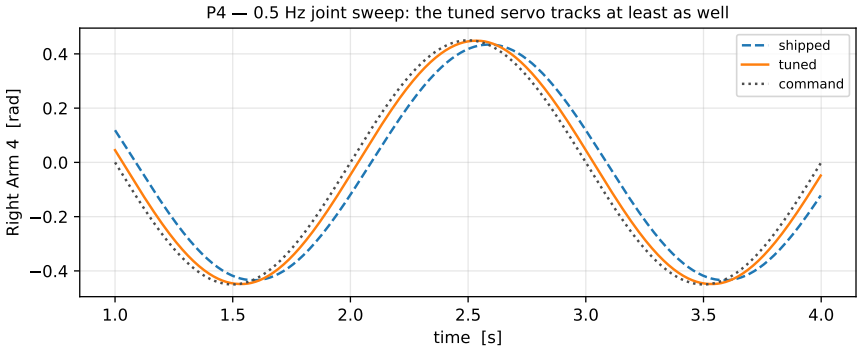


Figure 3: P4 — a 0.5 Hz joint sweep. The gain change is a latency change: the tuned trace sits on the command where the shipped one trails it.

lag was that damping over that gain, 100 ms, arrived at by accident. They are now tuned by Eq. (2) minus the joint's own damping. This is the joint the August palm-flip report is about (see `front_workspace_retargeting.tex`).

6 The real robot is a different regime, and is not changed here

On hardware the same law runs in the DaMiao MIT mode, but k_d is hard-capped at the wire limit $k_d \lesssim 5$. Combining that with Eq. (2) gives, for a target lag $L = k_d/k_p$,

$$\zeta \;=\; \tfrac{1}{2}\sqrt{k_d L / M_j}, \tag{3}$$

so at the cap, lag and damping trade directly and the achievable bandwidth is far below the model's. The shipped real-robot set (`control_param.json`, `arm.gains`: $k_p = 25$, $k_d = 2.5$ on the shoulder) is $L = 100$ ms at $\zeta = 0.44$, and it was deliberately softened on 2026-07-29 for a *compliance* request ("as compliant as possible"), with the gravity feed-forward carrying the load. Those gains are therefore **not changed by this work**: raising them is a hardware decision that needs an on-robot check with the operator present, and the compliance choice they encode is not ours to revert. A derived candidate set at the k_d cap is recorded in `control_param.json` under `arm.gains_pd_candidate` as documentation, explicitly not live.

7 Limitations

- M_j is a median over the joint's own range with the other arm joints sampled uniformly; the true reflected inertia also depends on the torso and lift pose, which are held at the home configuration here.
- The jitter model is white noise at the physics rate. Real retargeting jitter is band-limited and correlated across joints, so the reported torque is an upper bound on the noise-driven component.
- Overshoot is quoted as the worst joint over the seven; individual joints whose live inertia sits well below the median will see a lower effective ζ and can overshoot more than the table's 2.3%.

- Nothing here is validated on hardware. The sim actuator is a proxy for the DaMiao MIT law with a different gain scale and no wire-rate limit.

Reproducing

```
python3 controller/experiments/exp_pd_tuning.py --all       # P1-P4 + figures
python3 controller/experiments/exp_pd_tuning.py --apply     # write the MJCF
make -C controller/docs arm_pd_tuning.pdf
```

Raw metrics land in `controller/experiments/results/pdtune_p*.json`.

References

[1] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009. (Ch. 8: independent joint control, and why the closed-loop bandwidth is set by $\sqrt{k_p/M}$.)

[2] E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," *IROS*, 2012.

[3] MABEL controller subsystem, "A Minimal Task-Space Arm Controller for MABEL: Pure PD, and What Gravity Compensation Buys," `controller/docs/basic_controller.tex`, 2026.